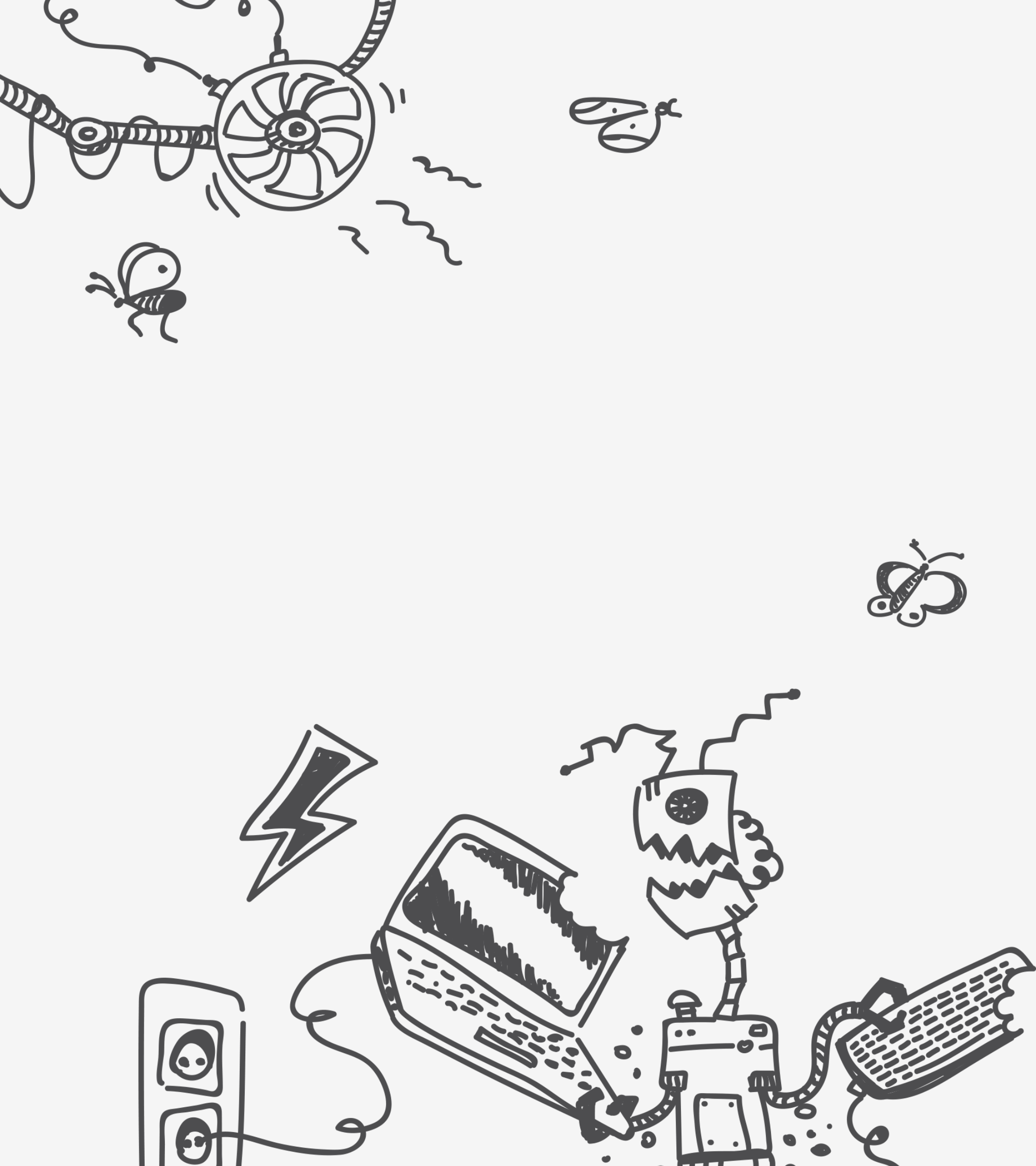
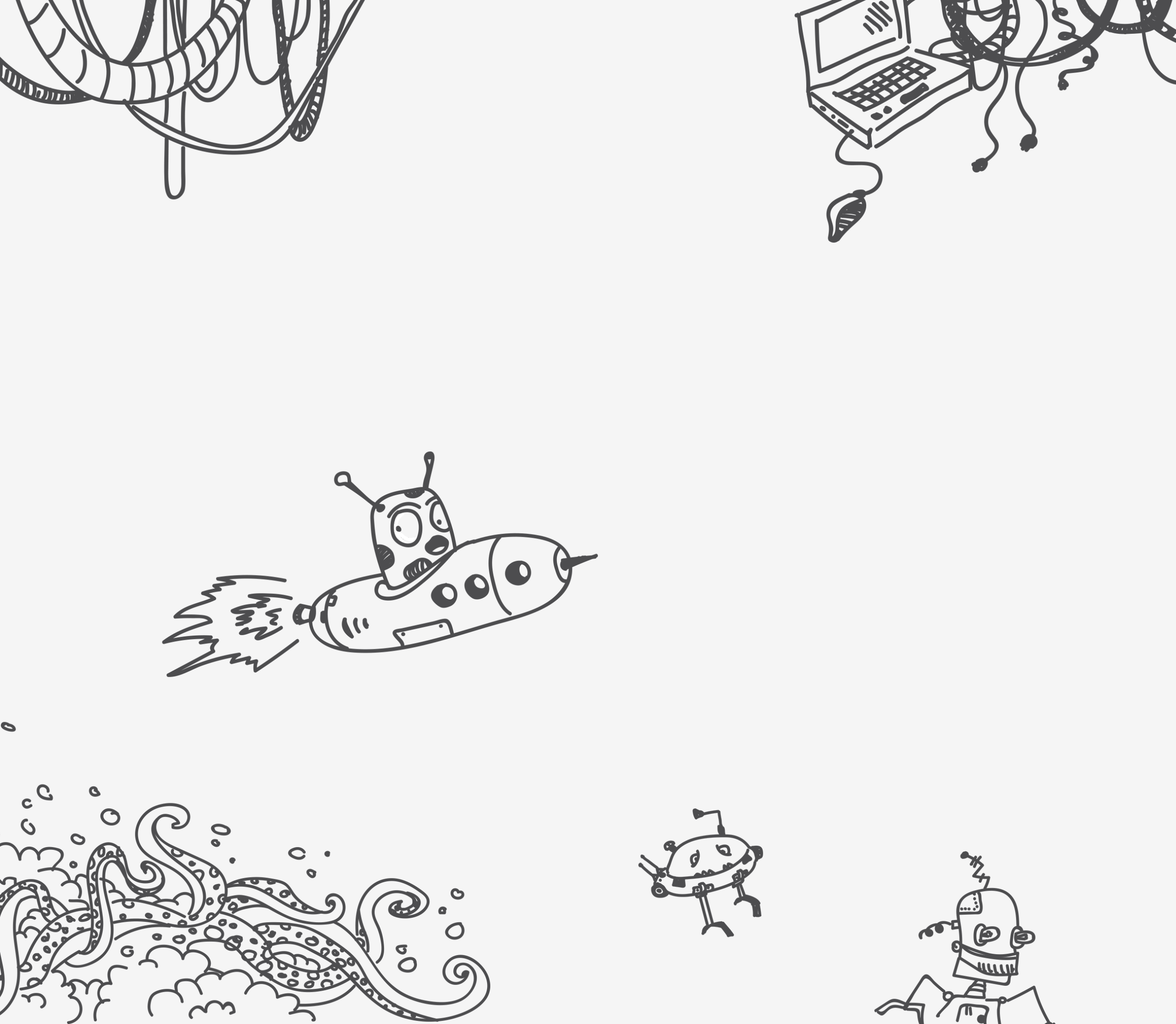




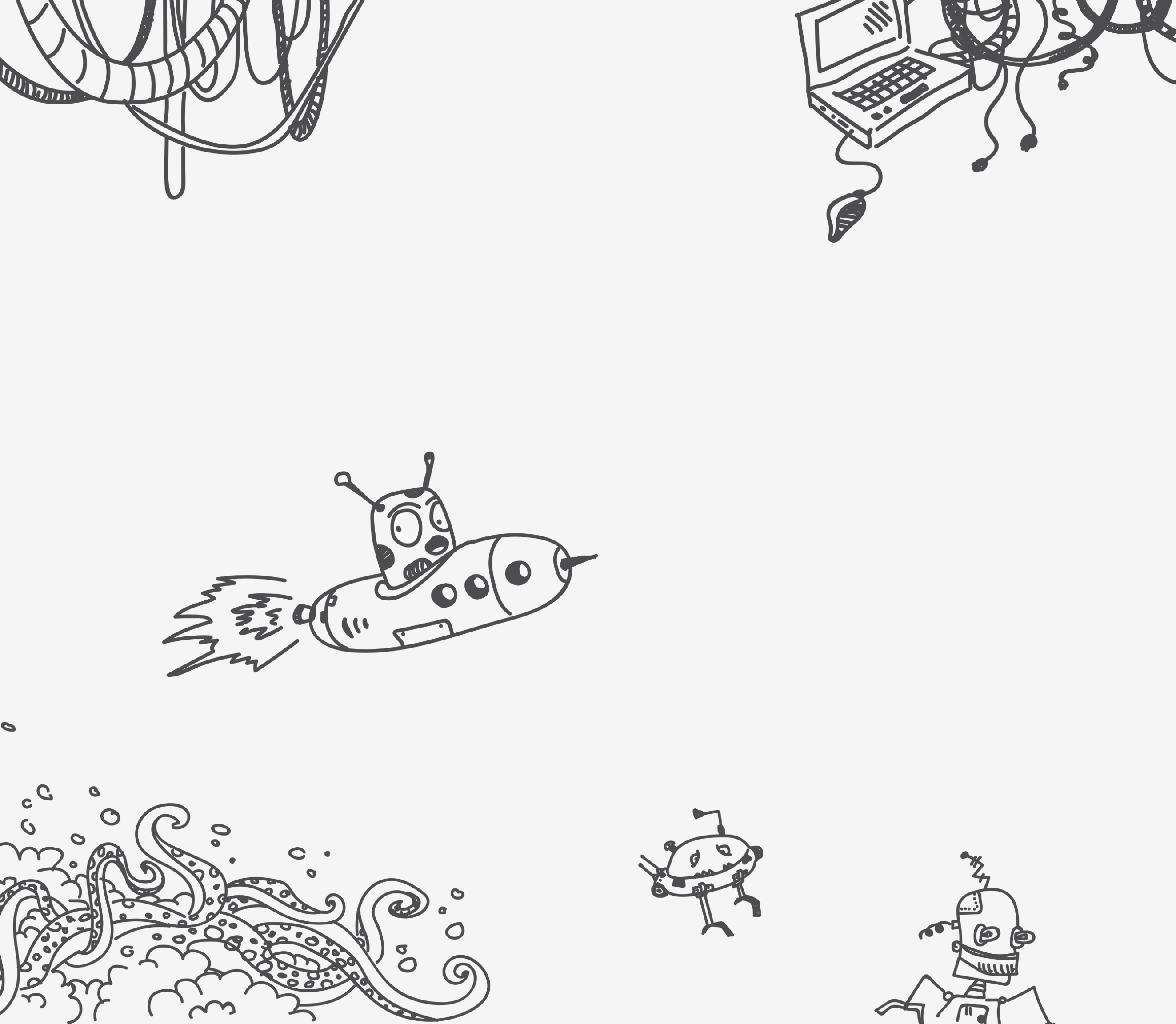
Linear regression - an example of implementation



Introduction

Introduction

A theoretical introduction to linear regression is behind us. We learned how the linear regression model works with a simple but illustrative example. Our goal was to determine the average fuel consumption of a car based on its average speed. Now let's move on to the practical part, in which we will build the model using Python and the **Scikit-learn** library we learned in the prework.



**What kind of problem
are we solving?**

What kind of problem are we solving?

Regression is a method that allows you to determine the relationship between variables. The goal of regression is to build a model that can predict the value of the dependent variable. The dependent variable in the case of regression is a continuous value, such as the amount of product sales, an assessment of the impact of advertising on customer behavior, a prediction of stock prices or the price of real estate.

We will be solving the last of these problems. We have a real estate dataset from King County, USA. The data contains information about properties located in the county, based on which we will be able to build a linear regression model predicting real estate prices.

Real estate prices in King County

To load the dataset, we use the `.read_csv` method from the Pandas library:

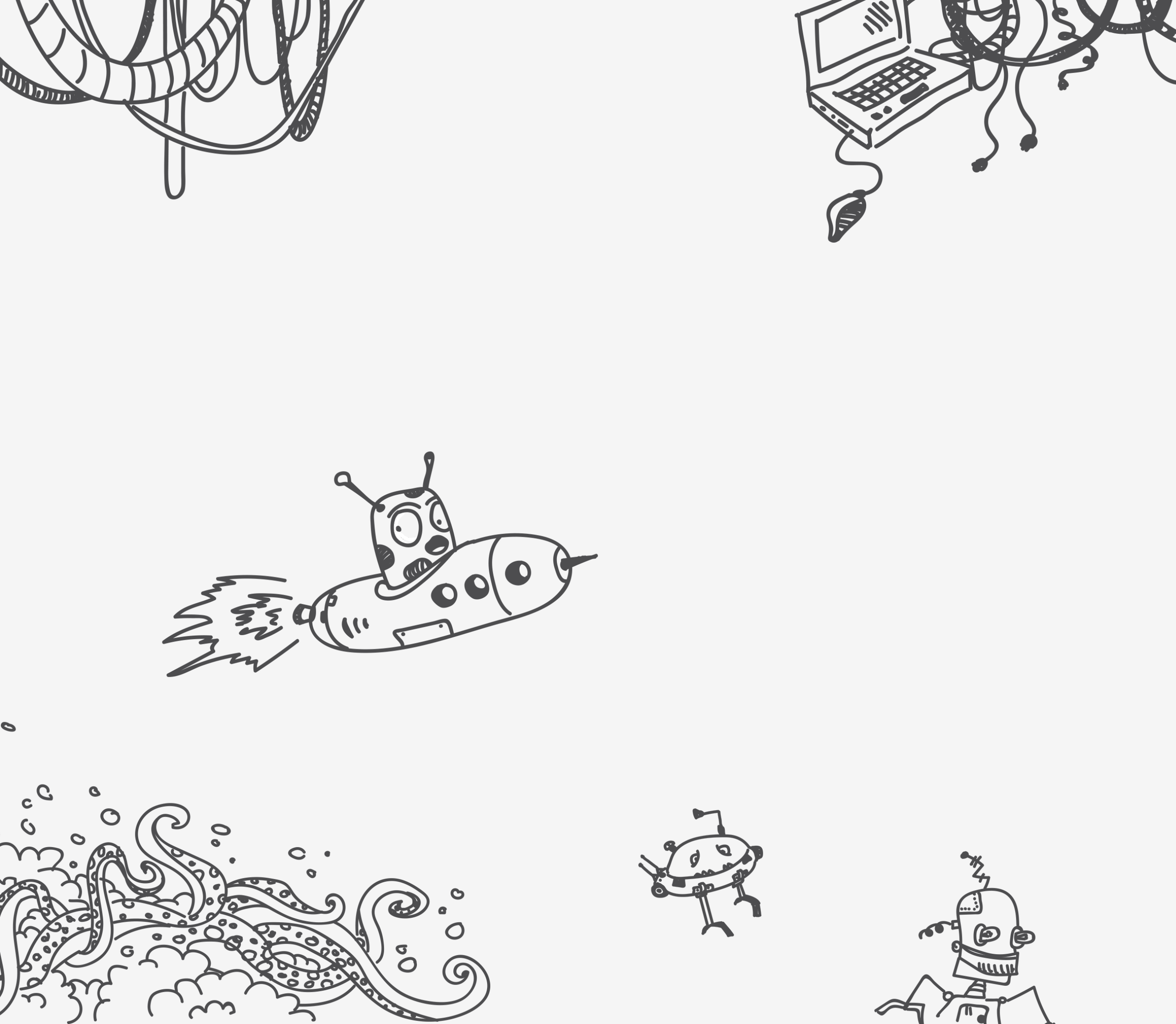
```
import pandas as pd  
  
housing_df = pd.read_csv("kc_house_data.csv")
```

Let's first get familiar with the data we will be working with:

```
housing_df.head(15)
```

Real estate prices in King County

- **id** - property identifier
- **date** - date the property was put up for sale
- **price** - price of the property (our dependent/explained variable)
- **bedrooms** - number of bedrooms
- **bathrooms** - number of bathrooms
- **sqft_living** - usable area of the house in square feet
- **sqft_lot** - area of the land lot in square feet
- **floors** - number of floors
- **waterfront** - information whether the house is on the waterfront
- **condition** - condition of the property expressed on a scale of 1 to 5
- **grade** - quality of construction (the higher the value, the better the quality, and the higher the cost of building the property)
- **sqft_above** - area of the house above ground level
- **sqft_basement** - area of the house below ground level
- **yr_built** - year of construction



Preparation of the dataset

Preparation of the dataset

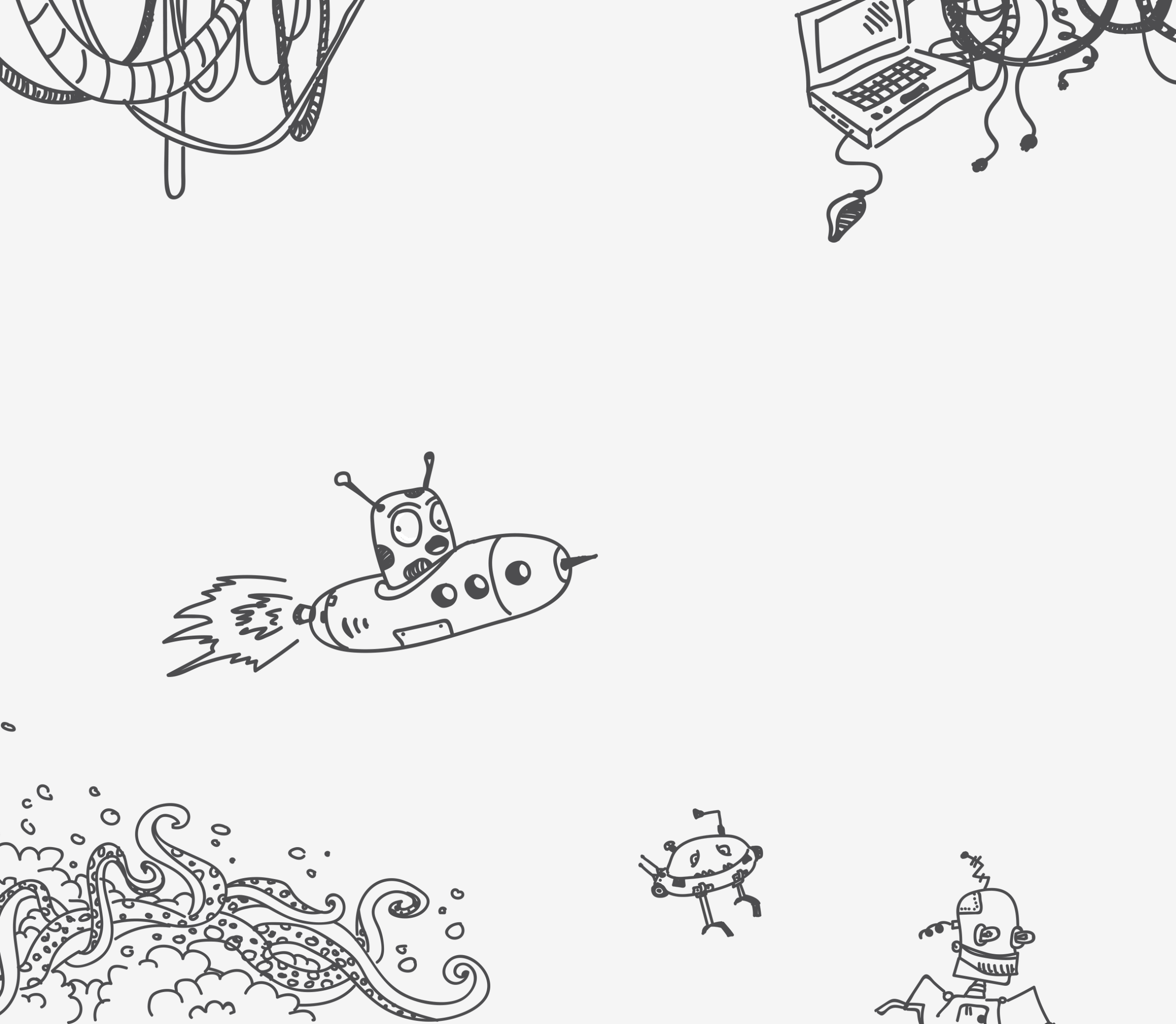
Let's check the dataset for empty values using the `.info()` method. We will also remove columns that are not relevant from the perspective of model construction.

```
# Check null values
housing_df.info()

# Drop uninformative columns
housing_df = housing_df.drop(["id", "date"], axis=1)
```

As we can see, there are no missing values in our data. In this presentation, we focus on discussing the implementation of the linear regression model in the Scikit-learn library, so let's now move on to splitting the data into a training set and a test set. We will not separate an additional validation set, because the linear regression model does not assume hyperparameters that we could optimize:

```
# Split data into train and test set
from sklearn.model_selection import train_test_split
```



Building the model

Building the model

After separating the training and test sets, we can move on to implementing the linear regression model. We import the LinearRegression class from the *linear_model module*. Next, we need to create our own instance of this class. We can declare it using the name *lin_reg*. The LinearRegression class does not require arguments, so a simple initialization of the object will suffice:

```
from sklearn.linear_model import LinearRegression

# Initialize linear regression model
lin_reg = LinearRegression()
```

As we mentioned in the prework, the classes of virtually all algorithms in the Scikit-learn library have two basic methods:

- fit - used to train the model,
- predict - used for prediction using the trained model.

Building the model

Let's focus at this point on the *.fit()* method. We need to train our model, that is, teach it to recognize patterns in our data. The method takes three arguments: **X**, **y** and **sample_weight**. The latter is responsible for introducing weights for individual samples, which allows us to tell the model that some of the observations are more important in training the model than others. E.g., in our case, data on properties with more than 5 rooms is more important than data on those with less than 5 rooms. However, we will not use this option and will fit our algorithm to the data using the first two arguments - (X and y):

```
lin_reg.fit(X_train, y_train)
```

Building the model

Great, we have trained our first regression model. We remember what the formula for linear regression looks like: $y = a_1x_1 + a_2x_2 + a_3x_3 \dots + b + e$ Our model received the training data, which has 18 variables, so we expect 19 coefficients (including the intercept **b**). Let's check their values using the corresponding attributes:

```
# Array of coefficients from a1 to a18. a1 stands for coefficient for "bedrooms", a2 - "bathroom"
# The order of the coefficients coincides with the order of the variables in the data set
print(f"Coefficients values: {lin_reg.coef_}")

# Intercept coefficient
print(f"Intercept coefficient value: {lin_reg.intercept_}")
```

Building the model

Therefore, the formula of our model can be presented as:

$$y = -34308 * \text{bedrooms} + 40312 * \text{bathrooms} + 112 * \text{sqft_living} + \dots - 6641646,71$$

The values of the coefficients standing next to the independent variables tell us how significant the influence of each variable is on the outcome of the model. The coefficient determines the direction (plus/minus sign) and strength of the relationship between the independent variable and the result (dependent variable). The higher the value of the coefficient, the stronger the relationship between the variable and the result.

Now let's check what results it gives - we will use the `.predict()` method for this, to which we will pass a set of variables from the test set:

```
# Predict y values based on features in test set
y_pred = lin_reg.predict(X_test)
```

Building the model

We can now check the effectiveness of the model against the default metric, the coefficient of determination R^2 . We call the `.score()` method twice on the model. Once for the training set, and a second time to check how our model performs on the test set. We could also import the **r2_score** method from the **metrics** module and there pass as argument the scores of our model (*y_pred*) and the actual labels from the test set (*y_test*).

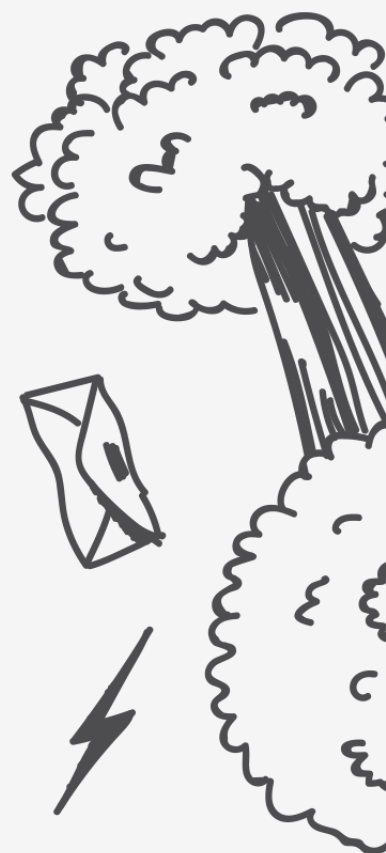
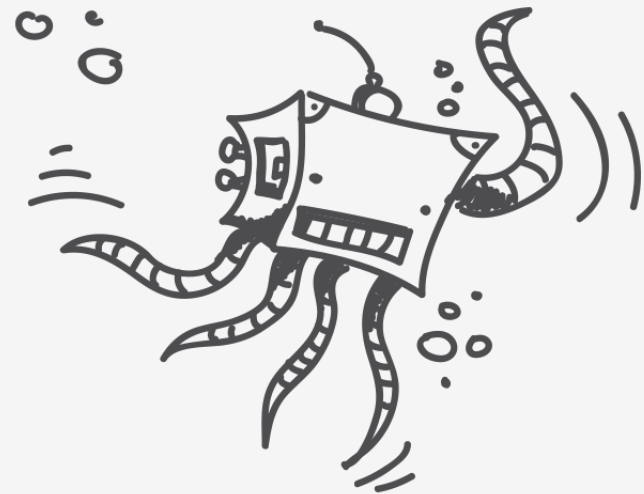
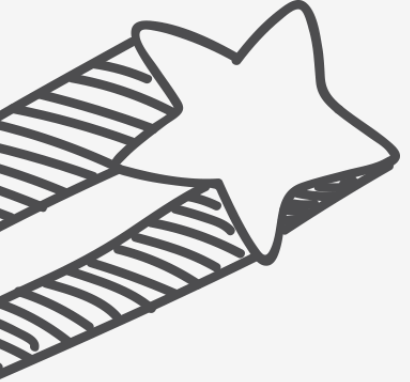
```
# Print R2 score value with .score() method
print(f"R2 score: {lin_reg.score(X_test, y_test)}") # or X_train, y_train for training set score

# Alternatively - R2 score using r2_score from sklearn.metrics
from sklearn.metrics import r2_score

print(r2_score(y_test, y_pred))
```

Summary

We've built the first model. We know how to use a powerful tool like the Scikit-learn library to train a linear regression model. We learned how to make a prediction and evaluate using a basic metric our model. In our example, which aims to predict the median price of real estate in California, we still have a lot of room for improvement. We will try to improve our model as we do the exercises. Good luck!



Practice time